



Classification on make_moons

- Generates dataset
- Builds Dataset / DataLoader
- Defines MLP model
- Trains with PyTorch training loop
- Evaluates on test set
- Plots decision boundary and training curves

```
In [13]: import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline

# -----
# Config / Hyperparameters
# -----
RANDOM_SEED = 42
BATCH_SIZE = 64
LR = 0.01
EPOCHS = 80
HIDDEN_UNITS = 64
TEST_SIZE = 0.25
NOISE = 0.25
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

torch.manual_seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)

# -----
# 1) Create dataset
# -----
X, y = make_moons(n_samples=1000, noise=NOISE, random_state=RANDOM_SEED)
X = X.astype(np.float32)
y = y.astype(np.int64)

# Standardize features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_SEED, stratify=y
)

# -----
```

```

# 2) PyTorch Dataset & DataLoader
# -----
class MoonDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.long)
    def __len__(self):
        return len(self.X)
    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

train_loader = DataLoader(MoonDataset(X_train, y_train), batch_size=BATCH_SIZE)
test_loader = DataLoader(MoonDataset(X_test, y_test), batch_size=BATCH_SIZE,

# -----
# 3) Define model
# -----
class MLP(nn.Module):
    def __init__(self, input_dim=2, hidden=HIDDEN_UNITS, num_classes=2, dropout):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden, hidden//2),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden//2, num_classes)
        )
    def forward(self, x):
        return self.net(x)

model = MLP().to(DEVICE)

# -----
# 4) Loss, optimizer
# -----
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LR)

# -----
# 5) Training loop
# -----
train_losses = []
train_accs = []
test_losses = []
test_accs = []

for epoch in range(1, EPOCHS+1):
    # --- train ---
    model.train()
    running_loss = 0.0
    correct = 0

```

```

total = 0
for xb, yb in train_loader:
    xb, yb = xb.to(DEVICE), yb.to(DEVICE)
    optimizer.zero_grad()
    logits = model(xb)
    loss = criterion(logits, yb)
    loss.backward()
    optimizer.step()

    running_loss += loss.item() * xb.size(0)
    preds = torch.argmax(logits, dim=1)
    correct += (preds == yb).sum().item()
    total += xb.size(0)

epoch_loss = running_loss / total
epoch_acc = correct / total
train_losses.append(epoch_loss)
train_accs.append(epoch_acc)

# --- eval on test set ---
model.eval()
running_loss = 0.0
correct = 0
total = 0
with torch.no_grad():
    for xb, yb in test_loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        logits = model(xb)
        loss = criterion(logits, yb)
        running_loss += loss.item() * xb.size(0)
        preds = torch.argmax(logits, dim=1)
        correct += (preds == yb).sum().item()
        total += xb.size(0)
test_loss = running_loss / total
test_acc = correct / total
test_losses.append(test_loss)
test_accs.append(test_acc)

if epoch % 10 == 0 or epoch==1 or epoch==EPOCHS:
    print(f"Epoch {epoch:03d}/{EPOCHS} | Train Loss: {epoch_loss:.4f} | Tr

# -----
# 6) Final evaluation
# -----
model.eval()
with torch.no_grad():
    X_test_t = torch.tensor(X_test, dtype=torch.float32).to(DEVICE)
    logits = model(X_test_t)
    preds = torch.argmax(logits, dim=1).cpu().numpy()
    acc = (preds == y_test).mean()
print(f"\nFinal Test Accuracy: {acc:.4f}")

# -----

```

```

# 7) Plot training curves
# -----
plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.plot(train_losses, label="Train Loss")
plt.plot(test_losses, label="Test Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.title("Loss Curve")

plt.subplot(1,2,2)
plt.plot(train_accs, label="Train Acc")
plt.plot(test_accs, label="Test Acc")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.title("Accuracy Curve")
plt.show()

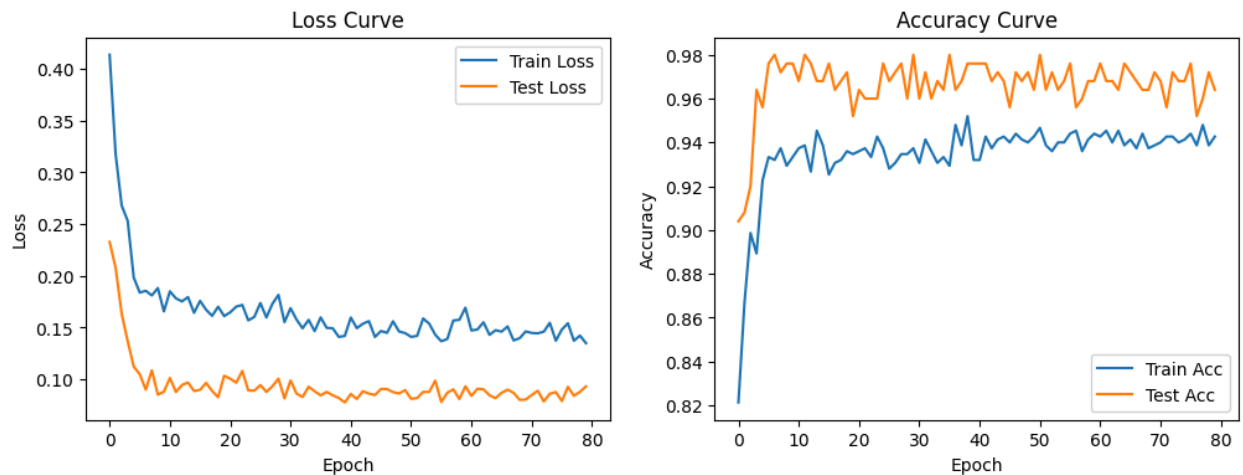
# -----
# 8) Decision boundary plot
# -----
def plot_decision_boundary(model, X, y, scaler=None, device='cpu', grid_steps=
    model.eval()
    X = np.array(X)
    x_min, x_max = X[:,0].min() - .5, X[:,0].max() + .5
    y_min, y_max = X[:,1].min() - .5, X[:,1].max() + .5
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, grid_steps), np.linspace(y_
    grid = np.c_[xx.ravel(), yy.ravel()].astype(np.float32)
    if scaler is not None:
        grid_scaled = scaler.transform(grid)
    else:
        grid_scaled = grid
    with torch.no_grad():
        grid_t = torch.tensor(grid_scaled, dtype=torch.float32).to(device)
        logits = model(grid_t)
        probs = torch.softmax(logits, dim=1)[:,1].cpu().numpy()
    Z = probs.reshape(xx.shape)
    plt.figure(figsize=(7,6))
    plt.contourf(xx, yy, Z, levels=50, cmap="RdBu", alpha=0.6)
    plt.scatter(X[:,0], X[:,1], c=y, cmap="bwr", edgecolor="k", s=25)
    plt.title(title)
    plt.xlabel("Feature 0")
    plt.ylabel("Feature 1")
    plt.colorbar(label="P(class=1)")
    plt.show()

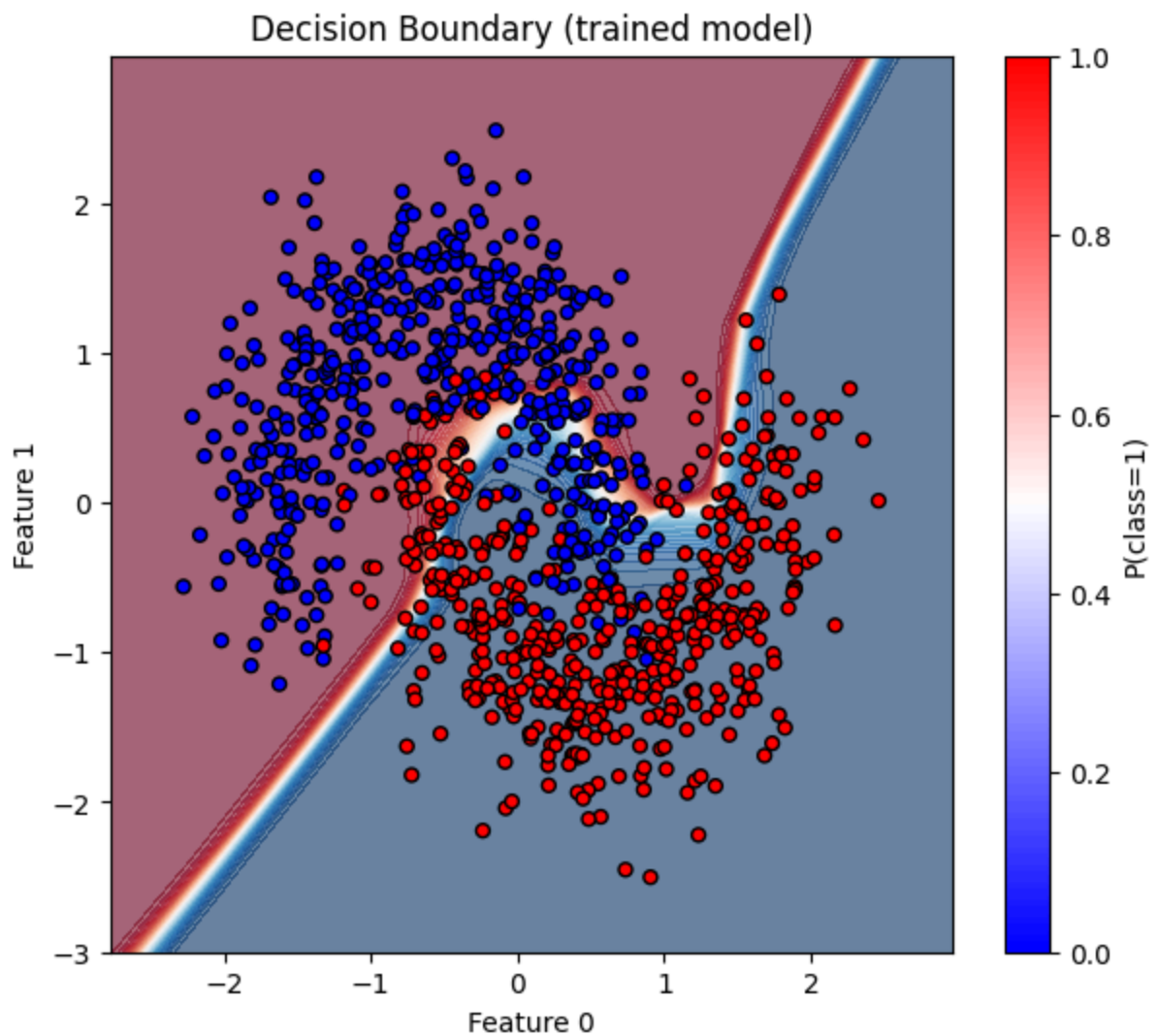
plot_decision_boundary(model, np.vstack([X_train, X_test]), np.hstack([y_train

```

Epoch 001/80 | Train Loss: 0.4137 | Train Acc: 0.8213 | Test Loss: 0.2328 | Test Acc: 0.9040
Epoch 010/80 | Train Loss: 0.1656 | Train Acc: 0.9333 | Test Loss: 0.0880 | Test Acc: 0.9760
Epoch 020/80 | Train Loss: 0.1610 | Train Acc: 0.9347 | Test Loss: 0.1034 | Test Acc: 0.9520
Epoch 030/80 | Train Loss: 0.1552 | Train Acc: 0.9373 | Test Loss: 0.0815 | Test Acc: 0.9800
Epoch 040/80 | Train Loss: 0.1419 | Train Acc: 0.9320 | Test Loss: 0.0776 | Test Acc: 0.9760
Epoch 050/80 | Train Loss: 0.1446 | Train Acc: 0.9427 | Test Loss: 0.0895 | Test Acc: 0.9640
Epoch 060/80 | Train Loss: 0.1692 | Train Acc: 0.9440 | Test Loss: 0.0931 | Test Acc: 0.9680
Epoch 070/80 | Train Loss: 0.1462 | Train Acc: 0.9387 | Test Loss: 0.0804 | Test Acc: 0.9720
Epoch 080/80 | Train Loss: 0.1349 | Train Acc: 0.9427 | Test Loss: 0.0932 | Test Acc: 0.9640

Final Test Accuracy: 0.9640





```
In [14]: # Predictions on Unknown Samples
def show_samples(model, X, y, scaler=None, n=10):
    model.eval()
    idx = np.random.choice(len(X), n, replace=False)
    Xs = X[idx]
    Ys = y[idx]

    # --- scale inputs if needed ---
    if scaler is not None:
        Xs_scaled = scaler.transform(Xs)
    else:
        Xs_scaled = Xs

    # --- predict ---
    with torch.no_grad():
        logits = model(torch.tensor(Xs_scaled, dtype=torch.float32).to(DEVICE))
        preds = torch.argmax(logits, dim=1).cpu().numpy()

    # --- print results ---
    print("\n===== SAMPLE PREDICTIONS =====\n")
```

```

    for i in range(n):
        print(f"Sample {i+1}:")
        print(f"   Input   : {Xs[i]}")
        print(f"   Predicted Class : {preds[i]}")
        print(f"   Actual Class   : {Ys[i]}")
        print("-----")

show_samples(model, np.vstack([X_train, X_test]), np.hstack([y_train, y_test]))

===== SAMPLE PREDICTIONS =====

Sample 1:
  Input   : [-1.4579552  0.18721378]
  Predicted Class : 0
  Actual Class   : 0
-----
Sample 2:
  Input   : [-0.41971424  0.81862795]
  Predicted Class : 0
  Actual Class   : 1
-----
Sample 3:
  Input   : [-0.01723179  1.2858678 ]
  Predicted Class : 0
  Actual Class   : 0
-----
Sample 4:
  Input   : [ 0.4926466 -0.6391626]
  Predicted Class : 1
  Actual Class   : 1
-----
Sample 5:
  Input   : [0.8312047 0.729598 ]
  Predicted Class : 0
  Actual Class   : 0
-----

```

Compare optimizers and loss variants on make_moons

```

In [15]: # Compare optimizers and loss variants on make_moons (PyTorch)
         # Run in a single cell in a notebook. Requires: torch, sklearn, matplotlib, nu

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

```

```

# -----
# Config
# -----
RANDOM_SEED = 42
BATCH_SIZE = 128
EPOCHS = 80
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
torch.manual_seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)

# -----
# Create dataset
# -----
X, y = make_moons(n_samples=2000, noise=0.25, random_state=RANDOM_SEED)
X = X.astype(np.float32)
y = y.astype(np.int64)

scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=RANDOM_SEED, stratify=y
)

class MoonDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.long)
    def __len__(self): return len(self.X)
    def __getitem__(self, idx): return self.X[idx], self.y[idx]

train_loader = DataLoader(MoonDataset(X_train, y_train), batch_size=BATCH_SIZE)
test_loader = DataLoader(MoonDataset(X_test, y_test), batch_size=BATCH_SIZE,

# -----
# Model (small MLP)
# -----
class SimpleMLP(nn.Module):
    def __init__(self, input_dim=2, hidden=64, dropout=0.2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden, hidden//2),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden//2, 2)
        )
    def forward(self, x): return self.net(x)

# -----
# Optional: Focal Loss implementation (small, for demonstration)

```



```

# -----
class FocalLoss(nn.Module):
    def __init__(self, gamma=2.0, reduction='mean'):
        super().__init__()
        self.gamma = gamma
        self.reduction = reduction
        self.ce = nn.CrossEntropyLoss(reduction='none')
    def forward(self, logits, targets):
        # logits: (N, C), targets: (N,)
        ce_loss = self.ce(logits, targets) # (N,)
        probs = torch.softmax(logits, dim=1)
        pt = probs[torch.arange(probs.size(0)), targets] # (N,)
        focal = (1 - pt) ** self.gamma * ce_loss
        if self.reduction == 'mean':
            return focal.mean()
        elif self.reduction == 'sum':
            return focal.sum()
        else:
            return focal

# -----
# Loss variants dictionary
# -----
loss_variants = {
    "CrossEntropy": lambda: nn.CrossEntropyLoss(),
    # Label smoothing: torch.nn.CrossEntropyLoss supports label_smoothing >=1.
    "LabelSmooth(0.1)": lambda: nn.CrossEntropyLoss(label_smoothing=0.1),
    "Focal(gamma=2)": lambda: FocalLoss(gamma=2.0)
}

# -----
# Optimizer configs to compare
# -----
optimizer_configs = {
    "SGD": lambda params: optim.SGD(params, lr=0.05),
    "SGD+Momentum": lambda params: optim.SGD(params, lr=0.05, momentum=0.9),
    "RMSprop": lambda params: optim.RMSprop(params, lr=0.01),
    "Adam": lambda params: optim.Adam(params, lr=0.01),
    "AdamW": lambda params: optim.AdamW(params, lr=0.01)
}

# -----
# Training & evaluation helpers
# -----
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        logits = model(xb)

```

```

        loss = criterion(logits, yb)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * xb.size(0)
        preds = logits.argmax(dim=1)
        correct += (preds == yb).sum().item()
        total += xb.size(0)
    return running_loss / total, correct / total

def eval_model(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for xb, yb in loader:
            xb, yb = xb.to(device), yb.to(device)
            logits = model(xb)
            loss = criterion(logits, yb)
            running_loss += loss.item() * xb.size(0)
            preds = logits.argmax(dim=1)
            correct += (preds == yb).sum().item()
            total += xb.size(0)
    return running_loss / total, correct / total

# -----
# Run experiments
# -----
results = {} # nested dict: results[loss_name][opt_name] -> record

for loss_name, loss_fn_builder in loss_variants.items():
    results[loss_name] = {}
    print(f"\n=== Loss variant: {loss_name} ===")
    for opt_name, opt_builder in optimizer_configs.items():
        print(f"-> Optimizer: {opt_name}")
        # fresh model per run
        model = SimpleMLP().to(DEVICE)
        criterion = loss_fn_builder()
        optimizer = opt_builder(model.parameters())

        history = {
            "train_loss": [], "train_acc": [],
            "test_loss": [], "test_acc": []
        }

        for epoch in range(1, EPOCHS+1):
            tr_loss, tr_acc = train_one_epoch(model, train_loader, criterion,
            te_loss, te_acc = eval_model(model, test_loader, criterion, DEVICE

            history["train_loss"].append(tr_loss)
            history["train_acc"].append(tr_acc)
            history["test_loss"].append(te_loss)
            history["test_acc"].append(te_acc)

```

```

        # print a few lines
        if epoch % 20 == 0 or epoch == 1 or epoch == EPOCHS:
            print(f" Epoch {epoch:03d} | TrLoss {tr_loss:.4f} TrAcc {tr_a

    results[loss_name][opt_name] = {
        "history": history,
        "final_test_acc": history["test_acc"][-1],
        "final_test_loss": history["test_loss"][-1]
    }

# -----
# Plotting comparison (loss + acc)
# -----
def plot_grid_for_loss(loss_name, results_for_loss):
    n_opts = len(results_for_loss)
    plt.figure(figsize=(14, 4))
    # Loss subplot
    plt.subplot(1, 2, 1)
    for opt_name, rec in results_for_loss.items():
        plt.plot(rec["history"]["train_loss"], label=f"{opt_name}-train", line
        plt.plot(rec["history"]["test_loss"], label=f"{opt_name}-test", linestyle
    plt.title(f"[{loss_name}] Loss curves")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend(fontsize=8)

    # Acc subplot
    plt.subplot(1, 2, 2)
    for opt_name, rec in results_for_loss.items():
        plt.plot(rec["history"]["train_acc"], label=f"{opt_name}-train", lines
        plt.plot(rec["history"]["test_acc"], label=f"{opt_name}-test", linestyle
    plt.title(f"[{loss_name}] Accuracy curves")
    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")
    plt.legend(fontsize=8)
    plt.tight_layout()
    plt.show()

for loss_name, res_for_loss in results.items():
    plot_grid_for_loss(loss_name, res_for_loss)

# -----
# Print summary table
# -----
import pandas as pd
rows = []
for loss_name, res_for_loss in results.items():
    for opt_name, rec in res_for_loss.items():
        rows.append((loss_name, opt_name, rec["final_test_loss"], rec["final_t
df = pd.DataFrame(rows, columns=["Loss", "Optimizer", "FinalTestLoss", "FinalT
print("\n=== Summary (final test loss & acc) ===")
print(df.sort_values(["Loss", "FinalTestAcc"], ascending=[True, False]).reset_

```

=== Loss variant: CrossEntropy ===

-> Optimizer: SGD

Epoch 001		TrLoss 0.6371	TrAcc 0.7047		TeLoss 0.5511	TeAcc 0.8880
Epoch 020		TrLoss 0.3103	TrAcc 0.8760		TeLoss 0.2668	TeAcc 0.8920
Epoch 040		TrLoss 0.2649	TrAcc 0.8980		TeLoss 0.2341	TeAcc 0.9080
Epoch 060		TrLoss 0.2308	TrAcc 0.9147		TeLoss 0.1986	TeAcc 0.9280
Epoch 080		TrLoss 0.1942	TrAcc 0.9240		TeLoss 0.1778	TeAcc 0.9340

-> Optimizer: SGD+Momentum

Epoch 001		TrLoss 0.5130	TrAcc 0.7713		TeLoss 0.3073	TeAcc 0.8780
Epoch 020		TrLoss 0.1709	TrAcc 0.9400		TeLoss 0.1639	TeAcc 0.9300
Epoch 040		TrLoss 0.1593	TrAcc 0.9440		TeLoss 0.1597	TeAcc 0.9360
Epoch 060		TrLoss 0.1530	TrAcc 0.9473		TeLoss 0.1621	TeAcc 0.9340
Epoch 080		TrLoss 0.1557	TrAcc 0.9440		TeLoss 0.1587	TeAcc 0.9360

-> Optimizer: RMSprop

Epoch 001		TrLoss 0.5740	TrAcc 0.7847		TeLoss 0.2354	TeAcc 0.8980
Epoch 020		TrLoss 0.1652	TrAcc 0.9427		TeLoss 0.1650	TeAcc 0.9360
Epoch 040		TrLoss 0.1502	TrAcc 0.9460		TeLoss 0.1728	TeAcc 0.9320
Epoch 060		TrLoss 0.1447	TrAcc 0.9487		TeLoss 0.1694	TeAcc 0.9300
Epoch 080		TrLoss 0.1439	TrAcc 0.9500		TeLoss 0.1653	TeAcc 0.9340

-> Optimizer: Adam

Epoch 001		TrLoss 0.4561	TrAcc 0.7840		TeLoss 0.2852	TeAcc 0.8840
Epoch 020		TrLoss 0.1497	TrAcc 0.9467		TeLoss 0.1618	TeAcc 0.9340
Epoch 040		TrLoss 0.1549	TrAcc 0.9467		TeLoss 0.1620	TeAcc 0.9280
Epoch 060		TrLoss 0.1449	TrAcc 0.9493		TeLoss 0.1596	TeAcc 0.9320
Epoch 080		TrLoss 0.1374	TrAcc 0.9493		TeLoss 0.1565	TeAcc 0.9360

-> Optimizer: AdamW

Epoch 001		TrLoss 0.4326	TrAcc 0.8013		TeLoss 0.2750	TeAcc 0.8940
Epoch 020		TrLoss 0.1571	TrAcc 0.9460		TeLoss 0.1668	TeAcc 0.9340
Epoch 040		TrLoss 0.1510	TrAcc 0.9460		TeLoss 0.1609	TeAcc 0.9340
Epoch 060		TrLoss 0.1500	TrAcc 0.9467		TeLoss 0.1660	TeAcc 0.9340
Epoch 080		TrLoss 0.1426	TrAcc 0.9493		TeLoss 0.1609	TeAcc 0.9320

=== Loss variant: LabelSmooth(0.1) ===

-> Optimizer: SGD

Epoch 001		TrLoss 0.6238	TrAcc 0.7647		TeLoss 0.5546	TeAcc 0.8820
Epoch 020		TrLoss 0.4081	TrAcc 0.8700		TeLoss 0.3728	TeAcc 0.8940
Epoch 040		TrLoss 0.3866	TrAcc 0.8860		TeLoss 0.3575	TeAcc 0.9080
Epoch 060		TrLoss 0.3658	TrAcc 0.9073		TeLoss 0.3420	TeAcc 0.9160
Epoch 080		TrLoss 0.3477	TrAcc 0.9187		TeLoss 0.3273	TeAcc 0.9260

-> Optimizer: SGD+Momentum

Epoch 001		TrLoss 0.5925	TrAcc 0.7153		TeLoss 0.3945	TeAcc 0.8860
Epoch 020		TrLoss 0.3250	TrAcc 0.9313		TeLoss 0.3122	TeAcc 0.9280
Epoch 040		TrLoss 0.3117	TrAcc 0.9380		TeLoss 0.3061	TeAcc 0.9320
Epoch 060		TrLoss 0.3081	TrAcc 0.9400		TeLoss 0.3078	TeAcc 0.9360
Epoch 080		TrLoss 0.3020	TrAcc 0.9447		TeLoss 0.3055	TeAcc 0.9360

-> Optimizer: RMSprop

Epoch 001		TrLoss 0.5988	TrAcc 0.7813		TeLoss 0.3489	TeAcc 0.9000
Epoch 020		TrLoss 0.3040	TrAcc 0.9433		TeLoss 0.3151	TeAcc 0.9340
Epoch 040		TrLoss 0.3080	TrAcc 0.9473		TeLoss 0.3156	TeAcc 0.9300
Epoch 060		TrLoss 0.3008	TrAcc 0.9467		TeLoss 0.3106	TeAcc 0.9340
Epoch 080		TrLoss 0.3067	TrAcc 0.9447		TeLoss 0.3127	TeAcc 0.9340

-> Optimizer: Adam

Epoch 001		TrLoss 0.4721	TrAcc 0.8340		TeLoss 0.3898	TeAcc 0.8940
Epoch 020		TrLoss 0.3022	TrAcc 0.9460		TeLoss 0.3065	TeAcc 0.9340

Epoch 040	TrLoss 0.2988	TrAcc 0.9473	TeLoss 0.3088	TeAcc 0.9340
Epoch 060	TrLoss 0.2990	TrAcc 0.9433	TeLoss 0.3066	TeAcc 0.9360
Epoch 080	TrLoss 0.2955	TrAcc 0.9493	TeLoss 0.3102	TeAcc 0.9360

-> Optimizer: AdamW

Epoch 001	TrLoss 0.4898	TrAcc 0.8073	TeLoss 0.3854	TeAcc 0.8820
Epoch 020	TrLoss 0.3014	TrAcc 0.9493	TeLoss 0.3082	TeAcc 0.9340
Epoch 040	TrLoss 0.2996	TrAcc 0.9473	TeLoss 0.3091	TeAcc 0.9340
Epoch 060	TrLoss 0.2965	TrAcc 0.9487	TeLoss 0.3157	TeAcc 0.9260
Epoch 080	TrLoss 0.2962	TrAcc 0.9480	TeLoss 0.3137	TeAcc 0.9300

=== Loss variant: Focal(gamma=2) ===

-> Optimizer: SGD

Epoch 001	TrLoss 0.1572	TrAcc 0.6947	TeLoss 0.1370	TeAcc 0.8700
Epoch 020	TrLoss 0.0826	TrAcc 0.8627	TeLoss 0.0725	TeAcc 0.8960
Epoch 040	TrLoss 0.0785	TrAcc 0.8793	TeLoss 0.0673	TeAcc 0.8920
Epoch 060	TrLoss 0.0709	TrAcc 0.8827	TeLoss 0.0624	TeAcc 0.9000
Epoch 080	TrLoss 0.0668	TrAcc 0.9027	TeLoss 0.0578	TeAcc 0.9140

-> Optimizer: SGD+Momentum

Epoch 001	TrLoss 0.1572	TrAcc 0.6213	TeLoss 0.1027	TeAcc 0.8600
Epoch 020	TrLoss 0.0531	TrAcc 0.9267	TeLoss 0.0442	TeAcc 0.9320
Epoch 040	TrLoss 0.0466	TrAcc 0.9380	TeLoss 0.0425	TeAcc 0.9280
Epoch 060	TrLoss 0.0429	TrAcc 0.9393	TeLoss 0.0421	TeAcc 0.9320
Epoch 080	TrLoss 0.0411	TrAcc 0.9433	TeLoss 0.0427	TeAcc 0.9320

-> Optimizer: RMSprop

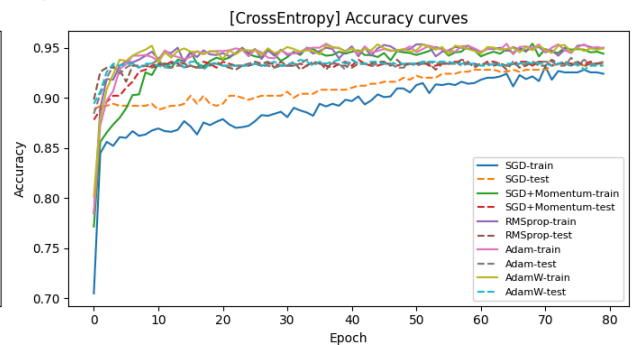
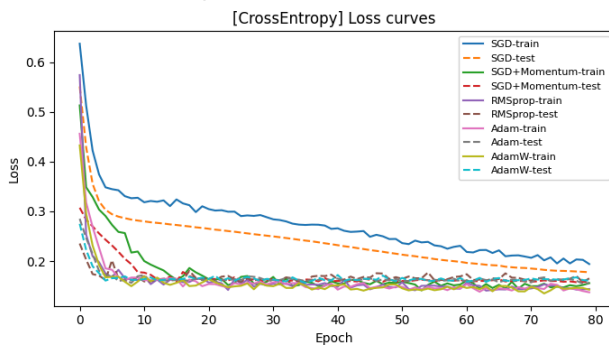
Epoch 001	TrLoss 0.1730	TrAcc 0.7520	TeLoss 0.0659	TeAcc 0.8980
Epoch 020	TrLoss 0.0461	TrAcc 0.9473	TeLoss 0.0429	TeAcc 0.9360
Epoch 040	TrLoss 0.0401	TrAcc 0.9447	TeLoss 0.0426	TeAcc 0.9320
Epoch 060	TrLoss 0.0366	TrAcc 0.9467	TeLoss 0.0418	TeAcc 0.9360
Epoch 080	TrLoss 0.0402	TrAcc 0.9493	TeLoss 0.0431	TeAcc 0.9300

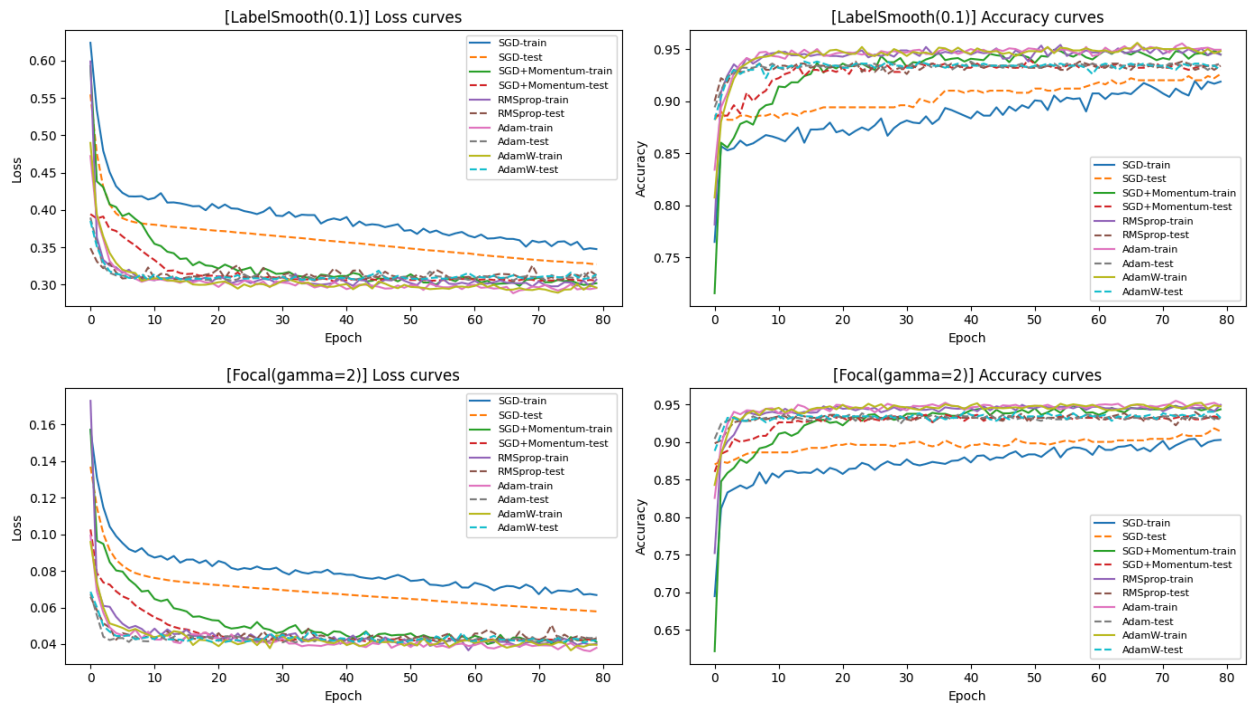
-> Optimizer: Adam

Epoch 001	TrLoss 0.0994	TrAcc 0.8253	TeLoss 0.0675	TeAcc 0.9040
Epoch 020	TrLoss 0.0435	TrAcc 0.9493	TeLoss 0.0437	TeAcc 0.9340
Epoch 040	TrLoss 0.0406	TrAcc 0.9427	TeLoss 0.0428	TeAcc 0.9340
Epoch 060	TrLoss 0.0390	TrAcc 0.9473	TeLoss 0.0427	TeAcc 0.9360
Epoch 080	TrLoss 0.0379	TrAcc 0.9480	TeLoss 0.0436	TeAcc 0.9320

-> Optimizer: AdamW

Epoch 001	TrLoss 0.0958	TrAcc 0.8427	TeLoss 0.0686	TeAcc 0.8880
Epoch 020	TrLoss 0.0411	TrAcc 0.9433	TeLoss 0.0412	TeAcc 0.9360
Epoch 040	TrLoss 0.0421	TrAcc 0.9440	TeLoss 0.0432	TeAcc 0.9400
Epoch 060	TrLoss 0.0421	TrAcc 0.9493	TeLoss 0.0417	TeAcc 0.9340
Epoch 080	TrLoss 0.0396	TrAcc 0.9473	TeLoss 0.0422	TeAcc 0.9320





=== Summary (final test loss & acc) ===

	Loss	Optimizer	FinalTestLoss	FinalTestAcc
0	CrossEntropy	SGD+Momentum	0.158717	0.936
1	CrossEntropy	Adam	0.156506	0.936
2	CrossEntropy	SGD	0.177756	0.934
3	CrossEntropy	RMSprop	0.165312	0.934
4	CrossEntropy	AdamW	0.160924	0.932
5	Focal(gamma=2)	SGD+Momentum	0.042710	0.932
6	Focal(gamma=2)	Adam	0.043633	0.932
7	Focal(gamma=2)	AdamW	0.042167	0.932
8	Focal(gamma=2)	RMSprop	0.043117	0.930
9	Focal(gamma=2)	SGD	0.057850	0.914
10	LabelSmooth(0.1)	SGD+Momentum	0.305516	0.936
11	LabelSmooth(0.1)	Adam	0.310169	0.936
12	LabelSmooth(0.1)	RMSprop	0.312663	0.934
13	LabelSmooth(0.1)	AdamW	0.313737	0.930
14	LabelSmooth(0.1)	SGD	0.327259	0.926

Compare different activation functions on make_moons

```
In [17]: # Compare different activation functions on make_moons (PyTorch)
# - trains same MLP architecture with different activations
# - plots loss & accuracy and decision boundaries
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```

import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

# -----
# Config
# -----
RSEED = 42
BATCH_SIZE = 128
EPOCHS = 80
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
LR = 0.01
NOISE = 0.25

torch.manual_seed(RSEED)
np.random.seed(RSEED)

# -----
# Data
# -----
X, y = make_moons(n_samples=1000, noise=NOISE, random_state=RSEED)
X = X.astype(np.float32)
y = y.astype(np.int64)

scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=RSEED, stratify=y
)

class MoonDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.long)
    def __len__(self): return len(self.X)
    def __getitem__(self, idx): return self.X[idx], self.y[idx]

train_loader = DataLoader(MoonDataset(X_train, y_train), batch_size=BATCH_SIZE)
test_loader = DataLoader(MoonDataset(X_test, y_test), batch_size=BATCH_SIZE,

# -----
# Model factory
# -----
class MLPFactory(nn.Module):
    def __init__(self, activation: nn.Module, hidden=64, dropout=0.2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, hidden),
            activation,
            nn.Dropout(dropout),
            nn.Linear(hidden, hidden//2),
            activation,

```

```

        nn.Dropout(dropout),
        nn.Linear(hidden//2, 2)
    )
    def forward(self, x): return self.net(x)

# -----
# Activations to test
# -----
activation_constructors = {
    "ReLU": lambda: nn.ReLU(),
    "Tanh": lambda: nn.Tanh(),
    "LeakyReLU": lambda: nn.LeakyReLU(negative_slope=0.01),
    "ELU": lambda: nn.ELU(alpha=1.0),
    "Softplus": lambda: nn.Softplus(),
}

# -----
# Training / eval helpers
# -----
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        logits = model(xb)
        loss = criterion(logits, yb)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * xb.size(0)
        preds = logits.argmax(dim=1)
        correct += (preds == yb).sum().item()
        total += xb.size(0)
    return running_loss / total, correct / total

def eval_model(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for xb, yb in loader:
            xb, yb = xb.to(device), yb.to(device)
            logits = model(xb)
            loss = criterion(logits, yb)
            running_loss += loss.item() * xb.size(0)
            preds = logits.argmax(dim=1)
            correct += (preds == yb).sum().item()
            total += xb.size(0)
    return running_loss / total, correct / total

```



```

# For decision boundary
def plot_decision_boundary(model, X, y, scaler=None, device='cpu', grid_steps=
    model.eval()
    X = np.array(X)
    x_min, x_max = X[:,0].min() - .5, X[:,0].max() + .5
    y_min, y_max = X[:,1].min() - .5, X[:,1].max() + .5
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, grid_steps), np.linspace(y_
    grid = np.c_[xx.ravel(), yy.ravel()].astype(np.float32)
    if scaler is not None:
        grid_scaled = scaler.transform(grid)
    else:
        grid_scaled = grid
    with torch.no_grad():
        grid_t = torch.tensor(grid_scaled, dtype=torch.float32).to(device)
        logits = model(grid_t)
        probs = torch.softmax(logits, dim=1)[:,1].cpu().numpy()
    Z = probs.reshape(xx.shape)
    plt.figure(figsize=(6,5))
    plt.contourf(xx, yy, Z, levels=50, cmap="RdBu", alpha=0.6)
    plt.scatter(X[:,0], X[:,1], c=y, cmap="bwr", edgecolor="k", s=18)
    plt.title(title)
    plt.xlabel("Feature 0")
    plt.ylabel("Feature 1")
    plt.colorbar(label="P(class=1)")
    plt.show()

# -----
# Run experiments
# -----
all_histories = {}
final_scores = {}
criterion = nn.CrossEntropyLoss()

for name, ctor in activation_constructors.items():
    print(f"\n=== Activation: {name} ===")
    torch.manual_seed(RSEED) # ensure same init per run for fairness
    np.random.seed(RSEED)
    model = MLPFactory(activation=ctor()).to(DEVICE)
    optimizer = optim.Adam(model.parameters(), lr=LR)
    history = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc":

    for epoch in range(1, EPOCHS+1):
        tr_loss, tr_acc = train_one_epoch(model, train_loader, criterion, opti
        te_loss, te_acc = eval_model(model, test_loader, criterion, DEVICE)
        history["train_loss"].append(tr_loss); history["train_acc"].append(tr_
        history["test_loss"].append(te_loss); history["test_acc"].append(te_ac
        if epoch % 20 == 0 or epoch==1 or epoch==EPOCHS:
            print(f"Epoch {epoch:03d} | TrLoss {tr_loss:.4f} TrAcc {tr_acc:.4f

    all_histories[name] = history
    final_scores[name] = (history["test_acc"][-1], history["test_loss"][-1])
# plot decision boundary for trained model (train set)
plot_decision_boundary(model, np.vstack([X_train, X_test]), np.hstack([y_t

```

```

scaler=scaler, device=DEVICE, title=f"Decision bound

# -----
# Plot comparison curves
# -----
plt.figure(figsize=(14,6))
plt.subplot(1,2,1)
for name, h in all_histories.items():
    plt.plot(h["train_loss"], label=f"{name}-train")
    plt.plot(h["test_loss"], linestyle='--', label=f"{name}-test")
plt.title("Loss curves")
plt.xlabel("Epoch"); plt.ylabel("Loss")
plt.legend(fontsize=8)

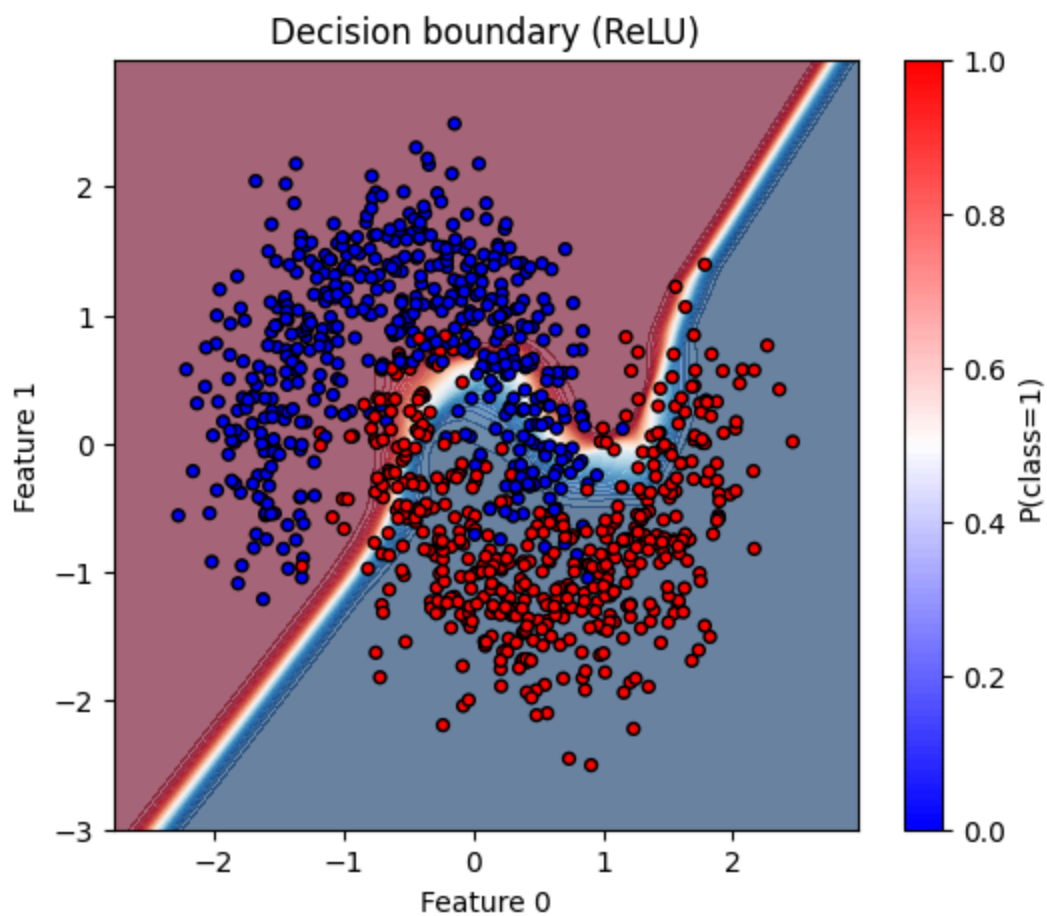
plt.subplot(1,2,2)
for name, h in all_histories.items():
    plt.plot(h["train_acc"], label=f"{name}-train")
    plt.plot(h["test_acc"], linestyle='--', label=f"{name}-test")
plt.title("Accuracy curves")
plt.xlabel("Epoch"); plt.ylabel("Accuracy")
plt.legend(fontsize=8)
plt.tight_layout()
plt.show()

# -----
# Print final summary
# -----
print("\nFinal test accuracies & losses:")
for name, (acc, loss) in final_scores.items():
    print(f"{name:10s} -> Test Acc: {acc:.4f} | Test Loss: {loss:.4f}")

```

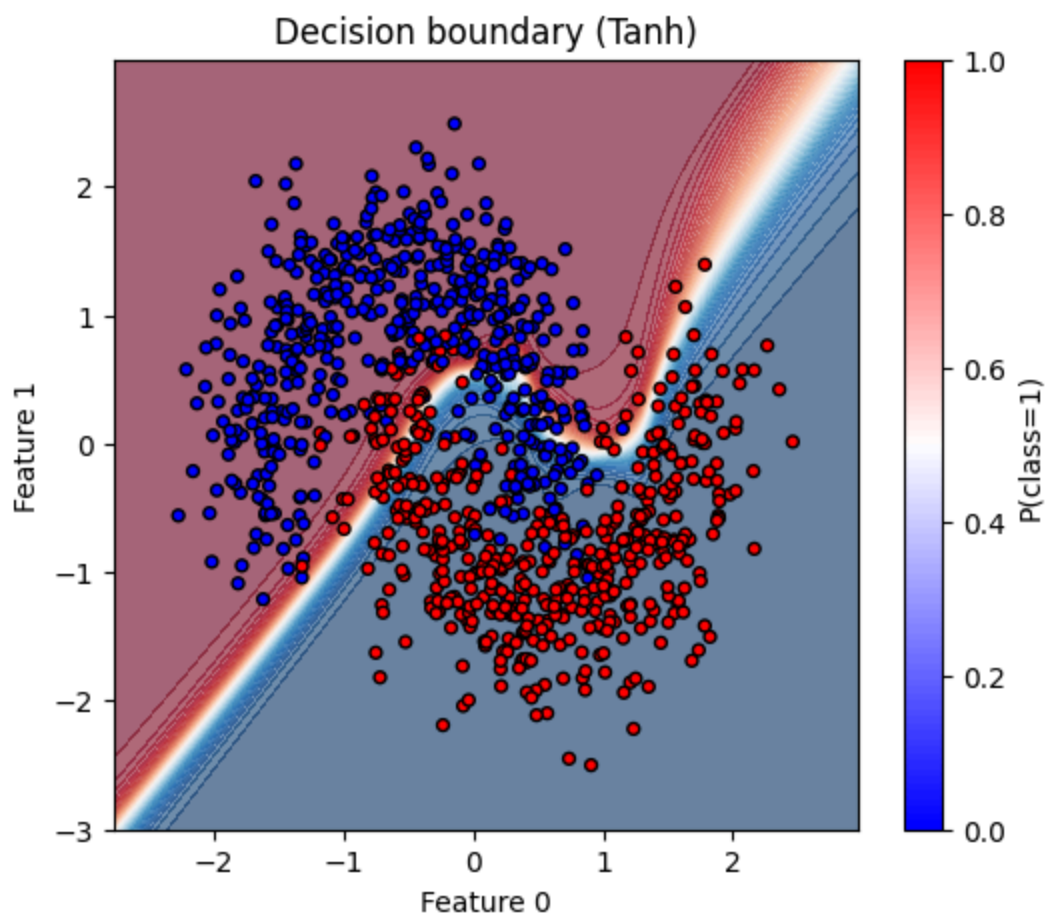
=== Activation: ReLU ===

Epoch 001		TrLoss 0.4905	TrAcc 0.7907		TeLoss 0.2430	TeAcc 0.9120
Epoch 020		TrLoss 0.1761	TrAcc 0.9267		TeLoss 0.0931	TeAcc 0.9640
Epoch 040		TrLoss 0.1542	TrAcc 0.9427		TeLoss 0.0842	TeAcc 0.9840
Epoch 060		TrLoss 0.1600	TrAcc 0.9373		TeLoss 0.0902	TeAcc 0.9640
Epoch 080		TrLoss 0.1412	TrAcc 0.9400		TeLoss 0.0872	TeAcc 0.9720



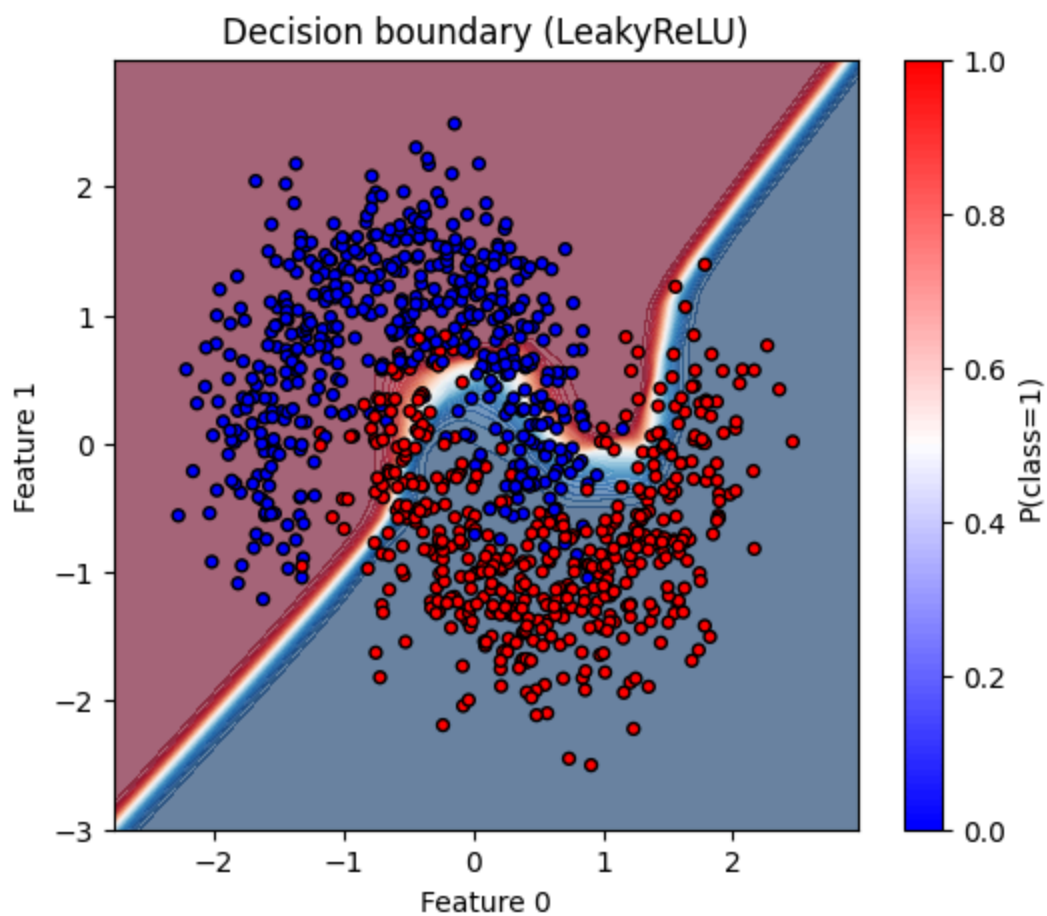
=== Activation: Tanh ===

Epoch 001		TrLoss 0.4257	TrAcc 0.7920		TeLoss 0.2481	TeAcc 0.9000
Epoch 020		TrLoss 0.1773	TrAcc 0.9293		TeLoss 0.0981	TeAcc 0.9720
Epoch 040		TrLoss 0.1773	TrAcc 0.9333		TeLoss 0.0943	TeAcc 0.9680
Epoch 060		TrLoss 0.1590	TrAcc 0.9440		TeLoss 0.0894	TeAcc 0.9680
Epoch 080		TrLoss 0.1734	TrAcc 0.9373		TeLoss 0.0803	TeAcc 0.9760



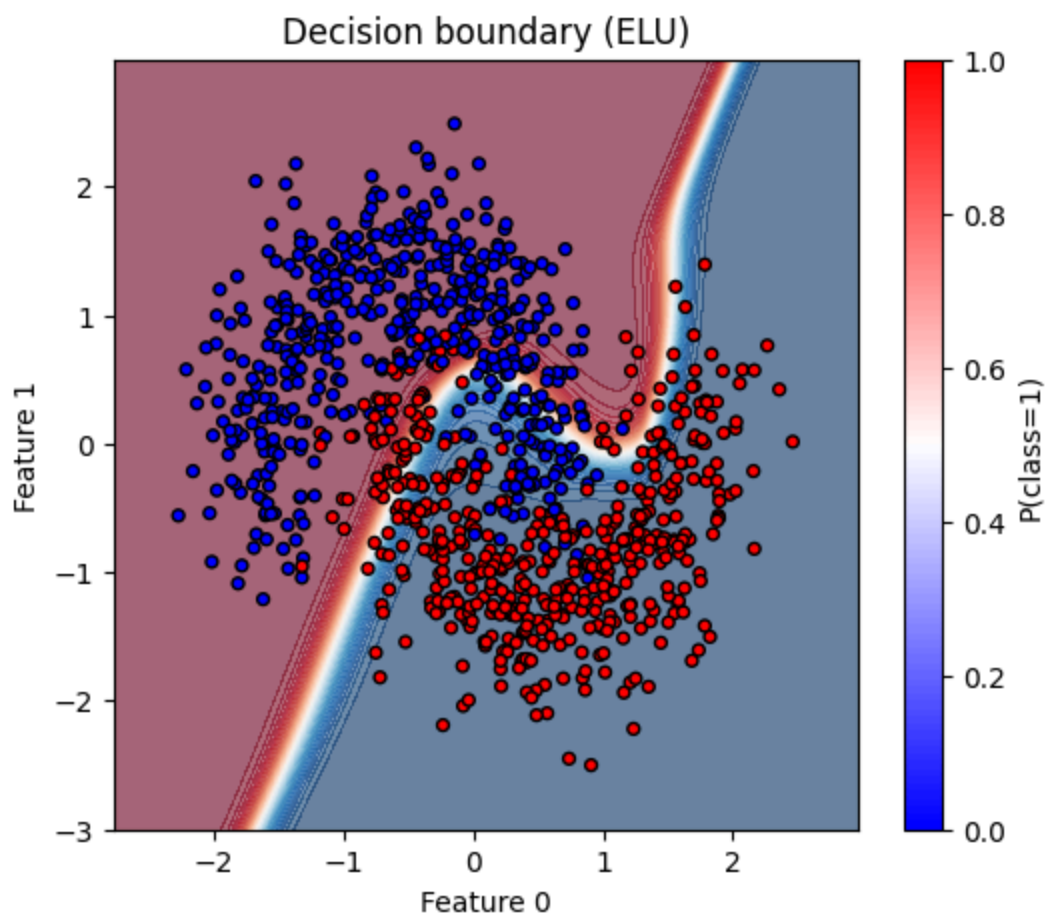
=== Activation: LeakyReLU ===

Epoch 001		TrLoss 0.4870	TrAcc 0.7880		TeLoss 0.2418	TeAcc 0.9120
Epoch 020		TrLoss 0.1741	TrAcc 0.9280		TeLoss 0.0911	TeAcc 0.9760
Epoch 040		TrLoss 0.1579	TrAcc 0.9400		TeLoss 0.0893	TeAcc 0.9640
Epoch 060		TrLoss 0.1548	TrAcc 0.9373		TeLoss 0.0888	TeAcc 0.9640
Epoch 080		TrLoss 0.1422	TrAcc 0.9360		TeLoss 0.0852	TeAcc 0.9840



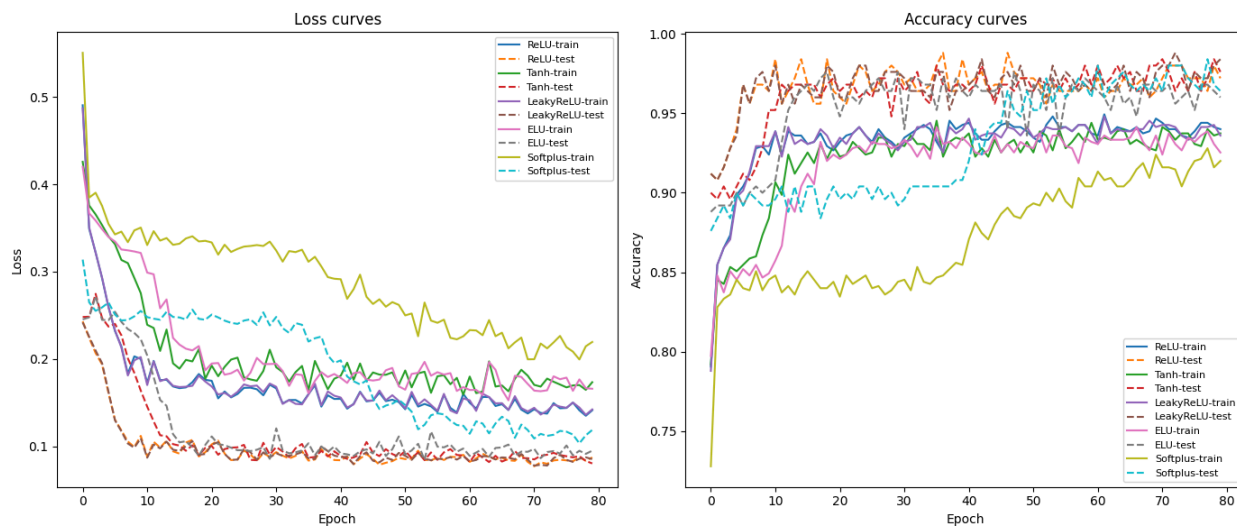
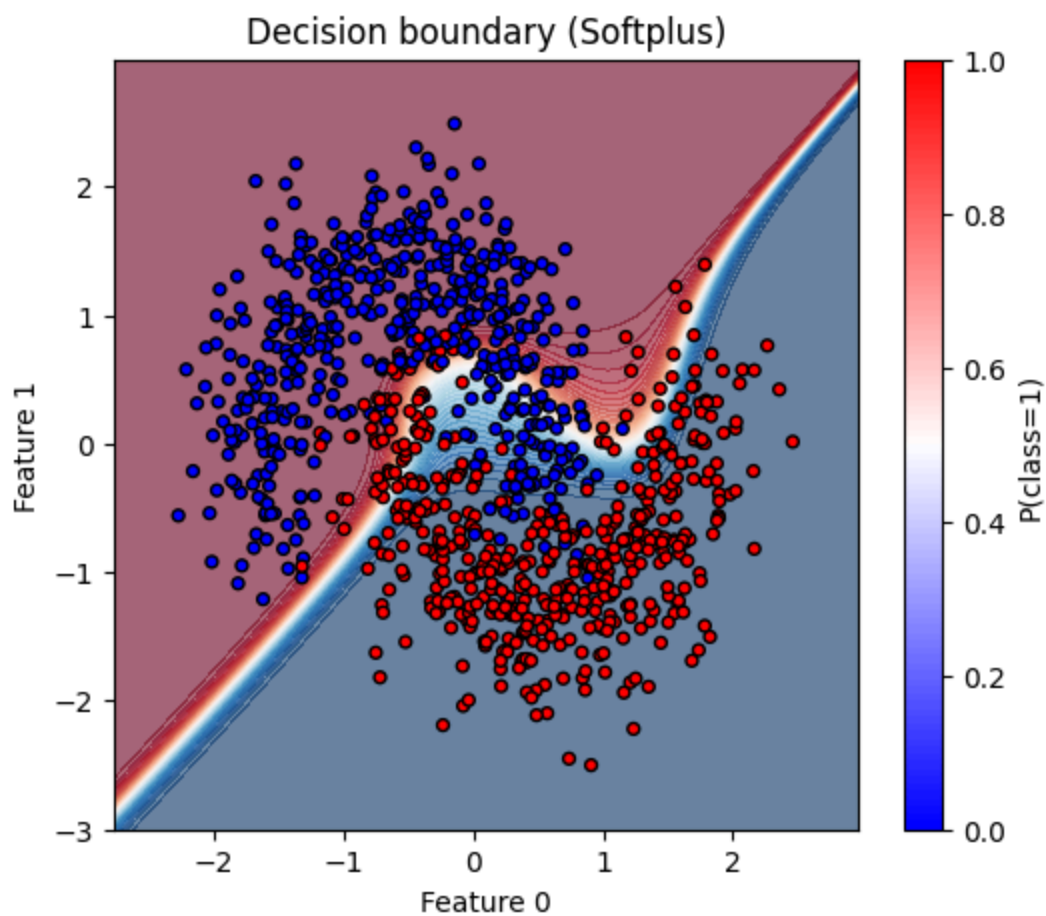
=== Activation: ELU ===

Epoch 001		TrLoss	0.4201	TrAcc	0.7973		TeLoss	0.2453	TeAcc	0.8880
Epoch 020		TrLoss	0.1869	TrAcc	0.9240		TeLoss	0.1015	TeAcc	0.9600
Epoch 040		TrLoss	0.1827	TrAcc	0.9347		TeLoss	0.0997	TeAcc	0.9640
Epoch 060		TrLoss	0.1677	TrAcc	0.9320		TeLoss	0.0974	TeAcc	0.9680
Epoch 080		TrLoss	0.1659	TrAcc	0.9253		TeLoss	0.0950	TeAcc	0.9600



=== Activation: Softplus ===

Epoch 001		TrLoss 0.5505	TrAcc 0.7280		TeLoss 0.3137	TeAcc 0.8760
Epoch 020		TrLoss 0.3352	TrAcc 0.8440		TeLoss 0.2449	TeAcc 0.9040
Epoch 040		TrLoss 0.2919	TrAcc 0.8547		TeLoss 0.1955	TeAcc 0.9080
Epoch 060		TrLoss 0.2259	TrAcc 0.9040		TeLoss 0.1235	TeAcc 0.9600
Epoch 080		TrLoss 0.2194	TrAcc 0.9200		TeLoss 0.1187	TeAcc 0.9640



Final test accuracies & losses:

ReLU	-> Test Acc: 0.9720	Test Loss: 0.0872
Tanh	-> Test Acc: 0.9760	Test Loss: 0.0803
LeakyReLU	-> Test Acc: 0.9840	Test Loss: 0.0852
ELU	-> Test Acc: 0.9600	Test Loss: 0.0950
Softplus	-> Test Acc: 0.9640	Test Loss: 0.1187

In []: